Qrisp Basics: The Programming Model

Raphael Seidel

IQM Quantum Computers

Eclipse Qrisp Community Day, Berlin, October 2026



Coding Session After the Talk

IQM

- After this talk there is a **30-minute coding session**: we simulate the double-slit experiment with Qrisp.
- Please **install now**, during the talk. Qrisp needs **Python 3.11 to 3.13**.

```
pip install qrisp jupyter
```

- The **notebooks** are in the repository behind the QR code.

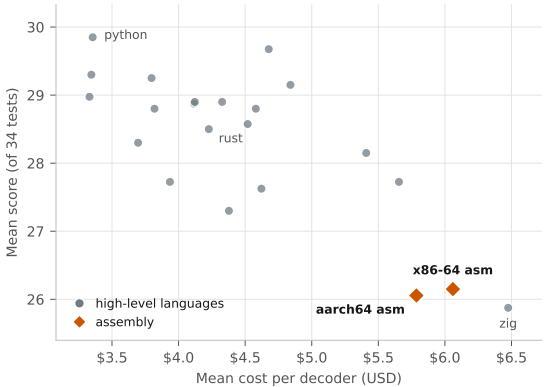


github.com/positr0nium/eqcd_26_basic_session

Motivation

Coding Agents Don’t Make Abstractions Obsolete

IQM



Medium effort; outliers J, Factor, Clojure omitted.

Data: D. Luu, danluu.com/pl-tokens (redrawn).

- AI agents (GPT-5.6) implement a **zstd decoder** from its RFC, in 22 languages.
- The high-level languages **cluster together**.
- **Assembly** costs ∼1.4× more and scores lower.

Quantum circuits are the **assembly language** of quantum computing.

What Is Qrisp?




```
pip install qrisp
```

qrisp.eu

- A **fully compilable**, high-level quantum programming framework.
- Central building block: the **QuantumVariable**, typed quantum data instead of wires.
- Built for software engineering: **prototyping, maintainability, testing, modularity, readability**.
- Compilable implementations of algorithms that **had none*** **before**: HHL, quantum backtracking, elliptic-curve Shor.

*Caveats apply, please ask me.

From Circuits to Variables

Why Not Just Write Circuits?

IQM

Quantum circuits as a programming model **prohibit collaborative software development**.

Two problems stand out:

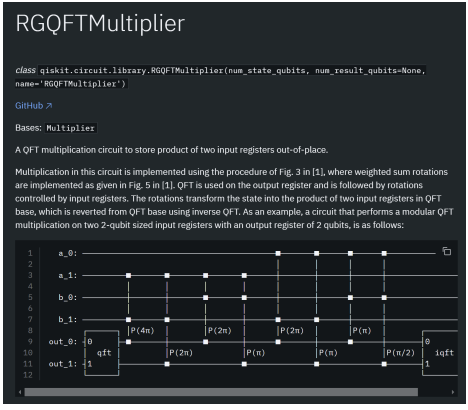
1. The **signature** problem: what does a subroutine expect, and what does it return?
2. The **modularization** problem: can I swap one implementation for another?



Wires don't scale as an abstraction.

Problem 1: Signatures

- Sharing a subroutine requires a well-documented **function signature**.
- A circuit's signature is a **list of qubit indices**. Which are inputs, outputs, ancillae? Signed? Fixed point?
- Qiskit's answer: **prose** in the docstring and a picture of the circuit.



Documentation of Qiskit's `RGQFTMultiplier`.

The Same Multiplication in Two Programming Models

IQM

Qiskit

```
from qiskit import QuantumCircuit, QuantumRegister
from qiskit.circuit.library import RGQFTMultiplier

a = QuantumRegister(n)
b = QuantumRegister(n)
res = QuantumRegister(2*n)
qc = QuantumCircuit(a, b, res)
for i in range(n):
    if 3 & 1<<i: qc.x(a[i])
for i in range(n):
    if 4 & 1<<i: qc.x(b[i])
qc.append(RGQFTMultiplier(n, 2*n),
          list(a) + list(b) + list(res))
```

qrisp

```
from qrisp import QuantumFloat

a = QuantumFloat(n)
b = QuantumFloat(n)
a[:] = 3
b[:] = 4
res = a * b

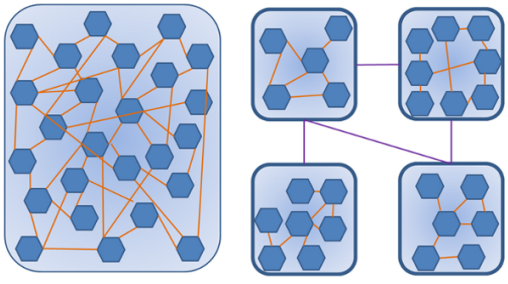
print(res)  # {12: 1.0}
```

	Qiskit	Qrisp
Typed result	✗	✓
Result allocated for you	✗	✓
Negative and non-integer values	✗	✓
Unequal input sizes	✗	✓

Problem 2: Modularity

- **Modular** code: subroutines are **exchangeable** and reusable in other algorithms.
- Circuit libraries can't be modular. Why? **Memory management**!
- Adders need **different ancilla counts**: Fourier 0, Cuccaro 1, Gidney n, QCLA $O(n)$. Swapping one forces the **caller** to manage the ancillae.

Modularity needs **automatic** qubit management.



QuantumVariables and Quantum Types

The QuantumVariable

IQM

```
from qrisp import *

qv = QuantumVariable(3)

h(qv[0])
cx(qv[0], qv[1])
cx(qv[0], qv[2])

print(qv)
# {'000': 0.5, '111': 0.5}
```

- The **constructor allocates** *n* qubits. No global circuit, no register bookkeeping.
- Gates act on **qubits** (`qv[0]`), **slices** or **whole variables** (`h(qv)`).
- `print(qv)` **simulates and decodes**: a dictionary of outcome probabilities. On hardware: `qv.get_measurement(backend=...)`.

Quantum Types: Bring Your Own Decoder

IQM

```python
from qrisp import *

class QuantumColor(QuantumVariable):

    def __init__(self):
        QuantumVariable.__init__(self, 2)

    def decoder(self, i):
        return ["red", "green", "blue", "yellow"][i]

# computational basis state prep
col_1 = QuantumColor()
col_1[:] = "blue"
print(col_1)   # {'blue': 1.0}

# arbitrary state prep
col_2 = QuantumColor()
col_2[:] = {"red": 1, "blue": 1}
print(col_2)   # {'red': 0.5, 'blue': 0.5}
```

- A quantum type is a **subclass** of QuantumVariable with its own `decoder`.
- The decoder maps the basis state $|i\rangle$ to a **label**. The encoder is derived from it, so `[:]` accepts a single label or a dictionary of amplitudes.
- Types make signatures **self-documenting**:
  `def oracle(c: QuantumColor) -> QuantumBool`

Built-in Quantum Types

IQM

Type	Represents	Highlights
QuantumFloat	signed/unsigned fixed-point numbers	`+ - * += -=`, comparisons, many adders
QuantumBool	truth values	`& | ~`, control and condition targets
QuantumChar	characters	`with q_ch == "a":`
QuantumModulus	integers mod N (Montgomery form)	modular `*=`, `+=`: the core of Shor

Algorithm modules add their own, e.g. the **one-hot integer** of quantum backtracking. Comparing it to a classical value (`h == 3`) costs a **single CNOT**.

QuantumFloat: Arithmetic in Superposition

IQM

```
from qrisp import *

a = QuantumFloat(3)        # 0, 1, ..., 7
b = QuantumFloat(3, -1)    # 0, 0.5, ..., 3.5
h(a)
b[:] = 1.5

c = a * b                  # QuantumFloat, 6 qubits
d = a >= 4                 # QuantumBool

print(multi_measurement([a, c, d]))
# {(0, 0.0, False): 0.125, (1, 1.5, False): 0.125,
#  (2, 3.0, False): 0.125, (3, 4.5, False): 0.125,
#  (4, 6.0, True): 0.125, (5, 7.5, True): 0.125, ...}
```

- `QuantumFloat(msize, exponent, signed)`: value = mantissa $\cdot 2^{\text{exponent}}$.
- In-place: `a += b`, `a -= 3`. Out-of-place: `a + b`, `a * b`, with the **result type inferred**.
- Comparisons <, >=, == return a **QuantumBool**.
- `multi_measurement`: the **joint** distribution of several variables.

QuantumArray: NumPy for Quantum Variables

IQM

- A NumPy-like container for many variables of one **qtype**.
- **Indexing, slicing**, `reshape`, `transpose` as in NumPy. Every entry is an ordinary QuantumVariable.
- Gates, arithmetic and environments act on **entries**: `qa[2] += 3`.
- `@` with a **classical matrix**: for QuantumModulus entries, the product is computed **modulo** *N*.

```
import numpy as np
from qrisp import *

qtype = QuantumModulus(7)
A = QuantumArray(qtype = qtype, shape=(2, 2))
A[:] = np.array([[1, 2], [3, 4]])
B = np.array([[1, 2], [3, 4]])

C = A @ B          # entries computed modulo 7

print(C)
# {OutcomeArray([[0, 3],
#                [1, 1]]): 1.0}

print(C[1, 0])     # {1: 1.0}
```

Quantum Memory Management

Allocation and Deallocation

- **Allocation**: call a QuantumVariable **constructor**.
- **Deallocation**, two options:
  1. `delete()`: you guarantee $|0\rangle$. `verify=True` checks it in simulation.
  2. `uncompute()`: Qrisp **reverts the computation**, then deletes.
- Freed qubits go back to the pool and are **reused** by later allocations.

```
from qrisp import *

# Allocation
a = QuantumFloat(10)
b = QuantumFloat(10)

# Do something
a[:] = 42
b += a

# Deallocation
b -= a          # manual uncomputation
b.delete(verify=True)
a.uncompute()   # automatic
```

Why Not Just Measure and Reset?

- Recycled qubits must be in $|0\rangle$.
- Measuring the ancilla in $\sum_i |i\rangle \left|f(i)\right\rangle$ **collapses** the entangled partner $|i\rangle$ as well.
- **Uncomputation** applies the inverse of the computation instead:

$$\sum_i |i\rangle \left|f(i)\right\rangle \longrightarrow \sum_i |i\rangle |0\rangle$$

- Often not optional but **required**: Grover's diffuser only works if the oracle's ancillae are disentangled again.



State-of-the-art tool to uncompute friendships.

Uncomputation is **garbage collection** for qubits.

Automatic Uncomputation

IQM

The uncompute method

```
from qrisp import *

def quadra_AND(ctrl, target):
    temp_0 = QuantumBool()
    mcx(ctrl[0:2], temp_0)

    temp_1 = QuantumBool()
    mcx(ctrl[2:4], temp_1)

    mcx([temp_0, temp_1], target)

    temp_0.uncompute()
    temp_1.uncompute()
```

The auto_uncompute decorator

```
from qrisp import *

@auto_uncompute
def quadra_AND(ctrl, target):
    temp_0 = QuantumBool()
    mcx(ctrl[0:2], temp_0)

    temp_1 = QuantumBool()
    mcx(ctrl[2:4], temp_1)

    mcx([temp_0, temp_1], target)
```

Uncomputes all **local** variables that are not returned.

Based on **Unqomp** (Paradis et al., PLDI 2021), generalized in Qrisp.

Quantum Environments

What Is a QuantumEnvironment?

IQM

- A `with` block whose content undergoes a **special compilation**.
- Think of them as **higher-order quantum functions**: they transform the code inside.
- They **nest** freely, and you can write your own.
- Built in: `control`, `invert`, `conjugate`, `gate_wrap`, ...

```
from qrisp import *

a = QuantumFloat(4)
b = QuantumBool()
h(b)

# Only executed where b is True
with control(b):
    a += 1

print(multi_measurement([b, a]))
# {(False, 0): 0.5, (True, 1): 0.5}
```

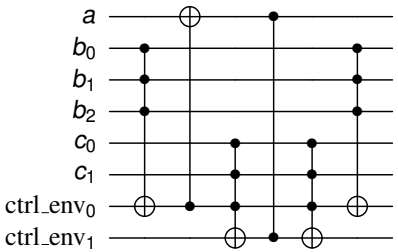
ControlEnvironment: Nested Controls

IQM

```
from qrisp import *

a = QuantumBool()
b = QuantumVariable(3)
c = QuantumVariable(2)

# Nested ControlEnvironments
with control([b[0], b[1], b[2]]):
    x(a)
    with control([c[0], c[1]]):
        z(a)
```



- The control condition is computed **once** into an ancilla.
- Every gate inside needs only a **single** control.

Custom Controlled Versions: `@custom_control`

IQM

```
from qrisp import *

@custom_control
def swap(a, b, ctrl=None):
    if ctrl is None:   # plain call
        cx(a, b)
        cx(b, a)
        cx(a, b)
    else:              # inside control(...)
        cx(a, b)
        mcx([ctrl, b], a)
        cx(a, b)

a = QuantumBool()
b = QuantumBool()
c = QuantumBool()
with control(c):
    swap(a, b)         # runs the else branch
```

- Inside `control`, every gate gets controlled. Often a **cheaper** controlled version exists.
- `@custom_control`: add a `ctrl` keyword. In a control environment it receives the **control qubit**.
- Used internally by the adders, modular arithmetic and `conjugate`.

Controlled swap	**T gates**	**CNOTs**
generic control	21	18
`@custom_control`	7	8

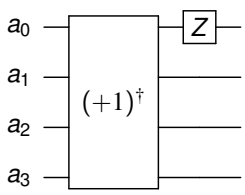
InversionEnvironment

- `with invert():` runs the **inverse** of its content: every gate inverted, in **reverse order**.
- Here: first `a -= 1`, then Z. Unsigned arithmetic wraps around, so $0 - 1 = 15$.
- Write the code for U once and get $U^\dagger$ for free, e.g. to undo a computation by hand.

```
from qrisp import *

a = QuantumFloat(4)
with invert():
    z(a[0])
    a += 1          # becomes a -= 1

print(a)            # {15: 1.0}
```



ConjugationEnvironment

IQM

- `with conjugate(V)(args):` applies V, then the block U, then $V^\dagger$: the operator $V^\dagger U V$.
- You write V once; Qrisp inserts $V^\dagger$ **automatically**.
- Here V is the QFT: in Fourier space, adding a constant is a layer of **phase gates** (Draper adder).

```python
import numpy as np
from qrisp import *

a = QuantumFloat(4)
c = 1
m = c * 2 * np.pi * 2**(-a.size)

# Draper adder: a += c in four lines
with conjugate(QFT)(a):
    for i in range(a.size):
        p(m * 2**i, a[i])

print(a)            # {1: 1.0}
```

The same pattern appears in Fourier adders, Grover diffusers and basis changes.

Controlling a Conjugation Is Cheap

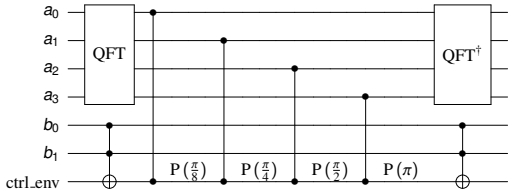
IQM

$$\text{control}(V^\dagger U V) \;=\; V^\dagger \,\text{control}(U)\, V \quad : \text{only } U \text{ needs the control.}$$

```
a = QuantumFloat(4)
b = QuantumFloat(2)
h(b)
c = 1
m = c * 2 * np.pi * 2**(-a.size)

with control(b):
    with conjugate(QFT)(a):
        for i in range(a.size):
            p(m * 2**i, a[i])

print(multi_measurement([b, a]))
# {(0, 0): 0.25, (1, 0): 0.25,
#  (2, 0): 0.25, (3, 1): 0.25}
```



A doubly controlled adder: the QFTs stay **uncontrolled**.

Putting It Together: Shor's Algorithm

Shor's Algorithm: Order Finding

IQM

```
from qrisp import *

def find_order(a, N):
    qg = QuantumModulus(N)
    qg[:] = 1
    qpe_res = QuantumFloat(2 * qg.size + 1,
                           exponent=-(2 * qg.size + 1))
    h(qpe_res)

    for i in range(len(qpe_res)):
        with control(qpe_res[i]):
            qg *= a
        a = (a * a) % N

    QFT(qpe_res, inv=True)
    return qpe_res.get_measurement()

print(find_order(7, 15))
```

- **QuantumModulus**: *= is an in-place **Montgomery** multiplication modulo N.
- **ControlEnvironment**: the multiplication becomes **controlled**.
- **Memory management**: the multiplier's ancillae are allocated and uncomputed for you.
- **Modularity**: swap the adder with one keyword, `inpl_adder=gidney_adder`.

Summary

- Circuits lack **signatures** and **modularity**. Qrisp programs with **typed QuantumVariables** instead of wires.
- **Quantum types** are QuantumVariables with a decoder: QuantumFloat, QuantumBool, QuantumModulus, QuantumArray, or your own.
- **Memory management**: allocate with a constructor, free with `delete`. Automatic uncomputation and qubit recycling.
- **Quantum environments** transform code blocks: `control`, `invert`, `conjugate`.
- Together they make **Shor** short, readable and modular.

Program the **algorithm**, let the compiler handle the **qubits**.

Up Next: Hands-On

IQM

The Double-Slit Experiment

- Positions as a **QuantumFloat**, slits via comparisons and **QuantumBools**.
- A slit oracle with `@auto_uncompute`, exact Grover and the inverse QFT produce the **interference pattern**.
- Bonus: a cheaper oracle, and your own diffuser with `conjugate`.

`double_slit_exercises.ipynb`



`github.com/positr0nium/eqcd_26_basic_session`

- Setup: `pip install qrisp jupyter`. Solutions are in `double_slit_solutions.ipynb`.
- Questions during and after the workshop: the **Qrisp Discord**, `discord.gg/v5np7DeBaq`.

qrisp.eu

Thanks!

```
pip install qrisp
```

